

Scaling

1. Overview

Scaling means increasing a system's ability to handle more users, more traffic, more data, or more operations without becoming slow or unavailable.

In system design interviews, scaling usually refers to how backend services, databases, caches, and storage systems grow when one machine is no longer enough.

Definition

Scaling is the process of increasing system capacity.

There are two main types:

Type	Meaning
Vertical scaling	Make one machine more powerful
Horizontal scaling	Add more machines

Related concepts:

Concept	Meaning
Stateless service	A service that does not store user/session-specific state locally
Caching	Store frequently used data in a faster layer
Replication	Keep copies of data or services across multiple machines
Sharding	Split data across multiple machines

Why It Matters

Without scaling, a system may fail when:

- More users join.
- More requests arrive per second.
- The database becomes too large.
- One server becomes a bottleneck.
- A machine crashes.

Scaling helps improve:

- Performance
- Availability
- Fault tolerance
- Throughput
- User experience

Where It Is Used In Real Systems

Scaling is used in:

- Backend servers handling API requests
- Databases storing large user data
- Caches like Redis and Memcached
- Search systems like Elasticsearch
- Messaging systems like Kafka
- Social media feeds
- E-commerce platforms
- Video streaming systems
- Payment systems

Why Interviewers Ask About It

Interviewers ask about scaling because it tests whether you can design systems beyond a single machine.

They want to know if you understand:

- Bottlenecks
- Load balancing
- Stateless services
- Database scaling
- Caching trade-offs
- Replication and consistency
- Sharding strategy
- Failure handling

Scaling is one of the most important system design topics for SDE placements.

2. Core Idea

The core idea is simple:

When one machine cannot handle the load, either make it stronger or distribute the load across multiple machines.

Intuition

Imagine a restaurant with too many customers.

There are two ways to serve more people:

Scaling Method	Restaurant Analogy
Vertical scaling	Hire a faster chef and buy a bigger stove
Horizontal scaling	Open more counters and hire more chefs

Vertical scaling improves one unit.

Horizontal scaling adds more units.

In modern distributed systems, horizontal scaling is usually preferred because it gives better fault tolerance and can grow gradually.

Small Example

Suppose one backend server can handle 1,000 requests per second.

Your app grows to 5,000 requests per second.

Options:

Option	Result
Vertical scaling	Upgrade the server to handle 5,000 RPS
Horizontal scaling	Use 5 servers, each handling around 1,000 RPS

With horizontal scaling, you also need a load balancer:

```
Users
|
v
Load Balancer
|
+---- Server 1
+---- Server 2
+---- Server 3
+---- Server 4
+---- Server 5
```

Step-By-Step Explanation

1. A system starts with one server and one database.
2. Traffic increases.
3. The server CPU, memory, network, or database becomes overloaded.
4. You identify the bottleneck.
5. For application servers, you add more servers behind a load balancer.
6. To make this work well, services should be stateless.
7. For frequently read data, you add caching.
8. For database read load, you add replicas.
9. For very large data or write load, you shard the database.
10. You monitor the system and keep improving bottlenecks.

3. Important Subtopics

3.1 Vertical Scaling

Vertical scaling means increasing the capacity of a single machine.

Examples:

- More CPU cores
- More RAM
- Faster SSD
- Bigger network bandwidth
- Stronger database server

Why It Matters

Vertical scaling is simple. You do not need to change much application logic.

It is useful when:

- The system is small or medium-sized.
- You need quick improvement.
- The software is difficult to distribute.
- The database has not reached its limits.

Example

A MySQL database is slow because it has only 8 GB RAM. You upgrade the machine to 64 GB RAM so more indexes and data fit in memory.

Common Interview Angle

Interviewers may ask:

Why not always use vertical scaling?

Expected answer:

Vertical scaling has limits. A single machine can only become so powerful. It is also expensive and creates a single point of failure.

3.2 Horizontal Scaling

Horizontal scaling means adding more machines to distribute load.

Examples:

- Add more backend servers.
- Add more database replicas.
- Add more cache nodes.
- Add more Kafka brokers.
- Add more search nodes.

Why It Matters

Horizontal scaling is the foundation of large distributed systems.

It helps with:

- Higher throughput
- Better availability
- Easier incremental growth
- Fault tolerance

Example

Instead of one API server handling all requests, use 10 API servers behind a load balancer.

Common Interview Angle

Interviewers may ask:

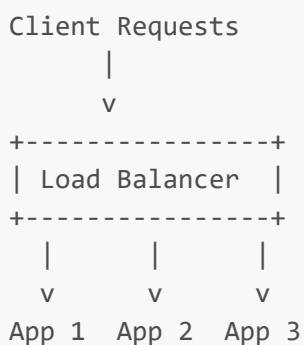
What changes are needed to horizontally scale an application server?

Expected answer:

Make the service stateless, put servers behind a load balancer, store shared state in external systems, and monitor health.

3.3 Load Balancing

Load balancing distributes incoming requests across multiple servers.



Why It Matters

Without a load balancer, clients would not know which server to call, and some servers may get overloaded while others stay idle.

Example

An NGINX or AWS Application Load Balancer routes user requests to healthy backend servers.

Common Interview Angle

Interviewers may ask about load balancing algorithms:

Algorithm	Meaning
Round robin	Send requests one by one to each server
Least connections	Send to server with fewest active connections
Weighted round robin	Send more traffic to stronger servers
IP hash	Route same client IP to same server

3.4 Stateless Services

A stateless service does not store user-specific session data in local server memory.

Each request should contain enough information to be processed independently, or the service should fetch required state from an external store.

Why It Matters

Stateless services are easy to horizontally scale.

If any server can handle any request, the load balancer can freely distribute traffic.

Example

Bad stateful design:

```
User logs in -> session stored in Server 1 memory  
Next request goes to Server 2 -> Server 2 does not know the user
```

Better stateless design:

```
User logs in -> token sent to client  
Every request carries token  
Any server can validate token
```

Or:

```
Session stored in Redis  
Any app server can read session from Redis
```

Common Interview Angle

Interviewers may ask:

Why are stateless services important for horizontal scaling?

Expected answer:

Because any request can go to any server. This improves scalability, fault tolerance, and deployment flexibility.

3.5 Caching

Caching means storing frequently accessed data in a faster storage layer to avoid expensive repeated work.

Common cache layers:

Layer	Example
Browser cache	Static files, images, CSS
CDN cache	Images, videos, public assets
Application cache	In-memory objects
Distributed cache	Redis, Memcached
Database cache	Buffer pool, query cache-like behavior

Why It Matters

Caching reduces:

- Database load
- Latency
- Computation cost
- Network calls

Example

An e-commerce product page is read frequently but updated rarely. Store product details in Redis.

```
Request product 101
  |
  v
Check Redis Cache
  |
  +-- Cache hit -> return quickly
  |
  +-- Cache miss -> query database -> store in cache -> return
```

Common Interview Angle

Interviewers may ask:

What can go wrong with caching?

Expected answer:

Stale data, cache invalidation issues, cache stampede, cache penetration, memory limits, and consistency problems.

3.6 Replication

Replication means keeping copies of data or services on multiple machines.

Database replication usually means one primary database and one or more replica databases.

```
Writes
|
v
Primary DB
|
+---- Replica DB 1
+---- Replica DB 2
+---- Replica DB 3
```

Why It Matters

Replication improves:

- Read scalability
- Fault tolerance
- Availability
- Disaster recovery

Example

All writes go to the primary database. Read-heavy requests, such as viewing profiles or product pages, go to replicas.

Common Interview Angle

Interviewers may ask:

Can replicas return stale data?

Expected answer:

Yes. If replication is asynchronous, replicas may lag behind the primary. This causes eventual consistency.

3.7 Sharding

Sharding means splitting data across multiple machines. Each machine stores only part of the data.

```
Users Table
|
+-- Shard 1: users 1 to 1M
```



```
+-- Shard 2: users 1M to 2M
+-- Shard 3: users 2M to 3M
```

Why It Matters

Replication helps read scaling, but sharding helps when:

- Data is too large for one machine.
- Write traffic is too high for one primary.
- Indexes no longer fit efficiently.
- Storage capacity becomes a bottleneck.

Example

A messaging app stores billions of messages. Instead of storing all messages in one database, it shards messages by `conversation_id`.

Common Interview Angle

Interviewers may ask:

How do you choose a shard key?

Expected answer:

A good shard key distributes data evenly, supports common queries, avoids hot shards, and minimizes cross-shard operations.

3.8 Partitioning vs Sharding

Partitioning is the general idea of splitting data into smaller parts.

Sharding usually means those parts are placed across different machines.

Concept	Meaning
Partitioning	Split data logically
Sharding	Split data across different database nodes

Example:

- Partitioning: One database has separate partitions for each month.
- Sharding: User data is split across multiple database servers.

3.9 Consistency

Consistency means whether all users see the latest correct data.

In scaled systems, consistency becomes harder because data may exist in caches, replicas, and shards.

Example

A user updates their profile picture. The database has the new picture, but CDN or Redis may still serve the old one for some time.

Common Interview Angle

Interviewers may ask:

Why does scaling make consistency harder?

Expected answer:

Because multiple copies of data may not update at exactly the same time.

4. Real-World Example

Example: Scaling A Food Delivery Backend

Suppose you are designing a food delivery app.

Initial version:

```
Mobile App -> Backend Server -> Database
```

This works for a few thousand users.

As the app grows:

1. Backend server CPU becomes overloaded.
2. Add more backend servers.
3. Put a load balancer in front.
4. Make backend services stateless.
5. Store sessions or tokens externally.
6. Add Redis cache for restaurant menus.
7. Add database replicas for read-heavy queries.
8. Shard orders by city or user ID when order volume grows.
9. Use monitoring to detect hot shards and slow queries.

Scaled version:

```
Users
|
v
CDN
|
v
Load Balancer
|
```

```

+---- App Server 1
+---- App Server 2
+---- App Server 3
    |
    v
  Redis Cache
    |
    v
  Primary DB
    |
+---- Read Replica 1
+---- Read Replica 2

```

When the database becomes too large:

```

Order Service
  |
+-- DB Shard 1: city = Delhi
+-- DB Shard 2: city = Mumbai
+-- DB Shard 3: city = Bengaluru

```

Important design decision:

Sharding by city may work well for food delivery because most queries are local to a city. But if one city has much more traffic, that shard may become hot.

5. Diagrams / Mental Models

Basic Single-Server System

```

Client
  |
  v
Server
  |
  v
Database

```

Problem:

The server and database are both single points of failure.

Horizontally Scaled App Servers

```

Clients
  |
  v

```

```

Load Balancer
|
+-- App Server A
+-- App Server B
+-- App Server C
|
v
Database

```

Cache-Aside Pattern

```

Application receives request
|
v
Check cache
|
+-- Hit -> return cached data
|
+-- Miss -> query DB
|
v
update cache
|
v
return data

```

Replication Mental Model

```

Write
|
v
Primary DB
/ | \
v v v
Replica1 Replica2 Replica3
^ ^ ^
| | |
Reads Reads Reads

```

Sharding Mental Model

```

Request for user_id = 87231
|
v
Shard router calculates: user_id % 3
|

```

v
Shard 0 / Shard 1 / Shard 2

Scaling Decision Flow

```
System is slow
  |
  v
Find bottleneck
  |
  +-- App CPU high? -> Add app servers + load balancer
  |
  +-- Repeated reads? -> Add cache
  |
  +-- DB reads high? -> Add read replicas
  |
  +-- DB writes/storage high? -> Shard
  |
  +-- One machine weak? -> Vertical scale temporarily
```

6. Common Interview Questions

1. What is scaling in system design?

Scaling means increasing a system's capacity to handle more traffic, data, or users.

Key points interviewer expects:

- Scaling can be vertical or horizontal.
- It applies to servers, databases, caches, queues, and storage.
- It improves performance and availability.

Common mistakes:

- Saying scaling only means adding servers.
- Ignoring database scaling.
- Not mentioning trade-offs.

2. What is vertical scaling?

Vertical scaling means upgrading a single machine with more CPU, RAM, storage, or network capacity.

Key points interviewer expects:

- Simple to implement.
- Has hardware limits.
- Can be expensive.
- Single point of failure may remain.

Common mistakes:

- Claiming vertical scaling is always bad.
- Forgetting that databases often use vertical scaling first.

3. What is horizontal scaling?

Horizontal scaling means adding more machines to distribute load.

Key points interviewer expects:

- Uses multiple servers.
- Usually requires load balancing.
- Works best with stateless services.
- Improves fault tolerance.

Common mistakes:

- Forgetting shared state problems.
- Assuming adding servers automatically fixes all bottlenecks.

4. Horizontal scaling vs vertical scaling?

Factor	Vertical Scaling	Horizontal Scaling
Method	Upgrade one machine	Add more machines
Complexity	Lower	Higher
Limit	Hardware limit	Can grow more
Fault tolerance	Usually weaker	Better
Cost pattern	Expensive high-end hardware	Commodity machines
Common use	Early stage, databases	Large-scale distributed systems

Key points interviewer expects:

- Vertical is simpler but limited.
- Horizontal is more scalable but needs distributed system design.

Common mistakes:

- Saying one is always better.
- Not connecting horizontal scaling with statelessness.

5. Why are stateless services important?

Stateless services are important because any server can handle any request.

Key points interviewer expects:

- Easier load balancing.

- Easier auto-scaling.
- Better fault tolerance.
- Sessions should be stored in Redis, database, or client tokens.

Common mistakes:

- Confusing stateless service with no database.
- Ignoring authentication/session handling.

6. What is caching?

Caching stores frequently accessed data in a faster layer to reduce latency and backend load.

Key points interviewer expects:

- Cache hit and cache miss.
- Reduces database load.
- Can introduce stale data.
- Cache invalidation is hard.

Common mistakes:

- Caching everything.
- Not setting TTL.
- Ignoring consistency issues.

7. What is a cache hit and cache miss?

A cache hit occurs when requested data is found in cache. A cache miss occurs when it is not found and must be fetched from the source, usually the database.

Key points interviewer expects:

- High cache hit ratio improves performance.
- Misses are more expensive.
- Cache warming can reduce cold-start misses.

Common mistakes:

- Not explaining what happens after a miss.
- Ignoring cache population.

8. What is replication?

Replication means maintaining copies of data on multiple machines.

Key points interviewer expects:

- Primary handles writes.
- Replicas can handle reads.
- Improves availability and read scalability.
- Replication lag can cause stale reads.

Common mistakes:

- Saying replication solves write scaling completely.
- Forgetting consistency problems.

9. What is sharding?

Sharding means splitting data across multiple database machines so each machine stores only a subset of the data.

Key points interviewer expects:

- Helps scale storage and writes.
- Requires shard key selection.
- Can cause cross-shard query complexity.
- Hot shards are a risk.

Common mistakes:

- Confusing sharding with replication.
- Choosing a bad shard key.

10. Replication vs sharding?

Replication copies the same data to multiple machines. Sharding splits different data across multiple machines.

Key points interviewer expects:

Concept	Replication	Sharding
Data layout	Same data copied	Data split
Helps with	Read scaling, availability	Storage and write scaling
Main risk	Stale replicas	Hot shards, cross-shard queries

Common mistakes:

- Saying replicas contain different data.
- Saying sharding always improves every query.

11. How do you choose a shard key?

A good shard key distributes data evenly and matches common query patterns.

Key points interviewer expects:

- High cardinality.
- Even distribution.
- Avoid hot partitions.
- Minimize cross-shard joins.
- Support important access patterns.

Common mistakes:

- Choosing timestamp as shard key for high-write systems.
- Choosing a low-cardinality key like country if most users are in one country.

12. What is a hot shard?

A hot shard is a shard that receives much more traffic than other shards.

Key points interviewer expects:

- Causes uneven load.
- Can happen due to bad shard key.
- Can happen due to celebrity users, popular products, or time-based keys.
- Can be solved using better partitioning, salting, or splitting.

Common mistakes:

- Thinking equal data size means equal traffic.
- Ignoring access pattern skew.

13. What is cache invalidation?

Cache invalidation is the process of removing or updating stale cached data when the original data changes.

Key points interviewer expects:

- TTL-based invalidation.
- Write-through or write-around patterns.
- Explicit delete/update on writes.
- Stale data trade-off.

Common mistakes:

- Assuming cache automatically stays correct.
- Not mentioning race conditions.

14. What happens when a replica lags?

Replica lag means a replica has not yet received the latest writes from the primary.

Key points interviewer expects:

- Reads from replica may be stale.
- Read-after-write consistency may break.
- Critical reads may need to go to primary.
- Monitoring lag is important.

Common mistakes:

- Assuming all replicas are always up to date.
- Not distinguishing synchronous and asynchronous replication.

7. Deep-Dive Questions

1. How would you scale a read-heavy system?

Use caching, database read replicas, CDN for static content, and possibly denormalized read models.

Clear answer:

Start by identifying repeated reads. Add a cache for hot data. Add read replicas to reduce load on the primary database. Use CDN for public static assets. Monitor cache hit rate and replica lag.

Important trade-off:

Read scaling often introduces stale data.

2. How would you scale a write-heavy system?

Use sharding, batching, asynchronous queues, partitioned logs, and optimized database indexes.

Clear answer:

Writes cannot be scaled only by adding read replicas. If one primary database is the bottleneck, split writes across shards using a good shard key. Use message queues for asynchronous processing where immediate consistency is not required.

Important trade-off:

Write scaling increases complexity in transactions, consistency, and query routing.

3. How do you handle read-after-write consistency with replicas?

Options:

- Read from primary immediately after write.
- Use sticky routing for a short period.
- Track replication lag.
- Use synchronous replication if latency allows.

Clear answer:

For critical user flows, read from the primary after a write. For less critical flows, allow eventual consistency and show slightly stale data.

4. How do you handle cache stampede?

Cache stampede happens when many requests miss the cache at the same time and all hit the database.

Solutions:

- Use request coalescing or locking.
- Add random jitter to TTLs.
- Refresh cache asynchronously.
- Serve stale data temporarily.

- Pre-warm cache for hot keys.

Clear answer:

Prevent all requests from recomputing the same value at once. Allow one request to rebuild the cache while others wait or receive stale data.

5. How do you reshard a large database?

Resharding means changing how data is distributed across shards.

Clear answer:

Create new shard mapping, migrate data gradually, dual-write or use change data capture, verify consistency, shift reads, then remove old mapping.

Key challenges:

- Data movement
- Downtime avoidance
- Correct routing
- Consistency during migration
- Backward compatibility

8. Comparison Tables

Horizontal Scaling vs Vertical Scaling

Feature	Horizontal Scaling	Vertical Scaling
Basic idea	Add more machines	Upgrade one machine
Complexity	Higher	Lower
Scalability limit	Higher	Limited by hardware
Fault tolerance	Better	Weaker if single machine
Cost	Scales incrementally	High-end machines are costly
Best for	Web servers, distributed databases, caches	Small systems, initial DB scaling
Interview keyword	Distributed load	Bigger box

Stateful vs Stateless Services

Feature	Stateful Service	Stateless Service
Stores local session?	Yes	No
Easy to scale horizontally?	Harder	Easier
Load balancing	May need sticky sessions	Any server can handle request
Failure impact	Local state may be lost	Less impact

Feature	Stateful Service	Stateless Service
Example	Server stores session in memory	JWT or Redis-backed session

Caching vs Replication

Feature	Caching	Replication
Purpose	Faster repeated access	Data availability and read scaling
Data stored	Usually hot subset	Usually full copy or replica set
Consistency	Often stale by design	Can be stale if async
Example	Redis product cache	MySQL read replica
Main risk	Invalid cache data	Replica lag

Replication vs Sharding

Feature	Replication	Sharding
Data distribution	Same data copied	Data split
Improves reads	Yes	Sometimes
Improves writes	Limited	Yes
Improves storage capacity	Limited	Yes
Complexity	Medium	High
Failure concern	Failover and lag	Routing and rebalancing

Cache Patterns

Pattern	How It Works	Use Case	Risk
Cache-aside	App checks cache, then DB	Common backend reads	Cache miss latency
Read-through	Cache loads data itself	Managed cache layer	More cache dependency
Write-through	Write cache and DB together	Stronger cache freshness	Higher write latency
Write-around	Write DB, load cache on read	Avoid unused cache writes	First read is slow
Write-back	Write cache first, DB later	High write throughput	Data loss risk

Shard Key Options

Shard Key	Advantage	Problem
<code>user_id</code>	Good for user-specific queries	Celebrity users can create hot shards
<code>conversation_id</code>	Good for chat messages	Large groups can become hot

Shard Key	Advantage	Problem
region	Good for locality	Uneven regions cause imbalance
timestamp	Easy for time queries	Recent shard may become hot
Hash of ID	Even distribution	Range queries become harder

9. Common Mistakes

- Thinking scaling only means adding more servers.
- Ignoring the database bottleneck.
- Confusing replication and sharding.
- Assuming cache always has correct data.
- Forgetting that horizontal scaling needs stateless services.
- Using sticky sessions as the first solution without understanding trade-offs.
- Choosing a shard key without considering query patterns.
- Ignoring hot keys and hot shards.
- Assuming read replicas solve write-heavy workloads.
- Not mentioning load balancers while discussing multiple app servers.
- Ignoring monitoring, metrics, and failure handling.
- Treating consistency problems as minor details.
- Overusing caching without invalidation strategy.
- Not considering operational complexity of sharding.

10. Edge Cases / Special Cases

10.1 Read-After-Write Problem

If a user updates data and immediately reads from a replica, they may see old data.

Solution:

- Read from primary after write.
- Use session consistency.
- Wait for replica catch-up for critical operations.

10.2 Cache Stampede

When a hot cache key expires, many requests may hit the database at once.

Solution:

- Locking
- TTL jitter
- Background refresh
- Serve stale data briefly

10.3 Cache Penetration

Requests repeatedly ask for data that does not exist, bypassing cache and hitting the database.

Solution:

- Cache negative results.
- Use Bloom filters.
- Validate inputs.

10.4 Cache Avalanche

Many cache keys expire at the same time, causing a sudden database spike.

Solution:

- Randomize TTLs.
- Pre-warm cache.
- Use rate limiting.

10.5 Hot Key

A single key receives too many requests.

Example:

- A viral post
- A celebrity profile
- A flash sale product

Solution:

- Replicate hot data.
- Split the key logically.
- Use local cache.
- Use CDN if public.

10.6 Hot Shard

One shard receives more traffic than others.

Solution:

- Change shard key.
- Add salting.
- Split the shard.
- Move hot tenants to dedicated shards.

10.7 Cross-Shard Query

A query needs data from multiple shards.

Problem:

It is slower and more complex.

Solution:

- Design shard key around common queries.
- Denormalize data.
- Use asynchronous aggregation.

10.8 Distributed Transactions

Transactions across shards are difficult.

Solution:

- Avoid cross-shard transactions when possible.
- Use eventual consistency.
- Use saga pattern.
- Use idempotent operations.

10.9 Sticky Sessions

Sticky sessions route the same user to the same server.

Useful when:

- Legacy stateful service stores session locally.

Problem:

- Reduces flexibility.
- Server failure can lose session.
- Load may become uneven.

10.10 Over-Sharding Too Early

Sharding too early adds unnecessary complexity.

Better approach:

1. Optimize queries.
2. Add indexes.
3. Add caching.
4. Add replicas.
5. Shard when storage/write limits require it.

11. How to Explain in Interview

Scaling means increasing a system's ability to handle more users, requests, or data. Vertical scaling upgrades one machine, while horizontal scaling adds more machines. For application servers, horizontal scaling usually needs a load balancer and stateless services so any server can handle any request. For performance, we use caching to reduce repeated database reads. For databases, replication helps scale reads and improve availability, while sharding splits data across machines to scale storage and writes. The main trade-offs are consistency, complexity, hot spots, and failure handling.

12. Quick Revision Notes

Key Definitions

Term	Definition
Scaling	Increasing system capacity
Vertical scaling	Upgrading one machine
Horizontal scaling	Adding more machines
Stateless service	Service that does not store request/session state locally
Cache	Fast layer for frequently accessed data
Replication	Copying data to multiple machines
Sharding	Splitting data across machines
Load balancer	Component that distributes traffic
Hot shard	Shard with too much traffic
Replica lag	Delay between primary and replica updates

Important Points

- Horizontal scaling is common for backend servers.
- Stateless services make horizontal scaling easier.
- Caching reduces latency and database load.
- Replication improves read scalability and availability.
- Sharding improves storage and write scalability.
- Replication copies data; sharding splits data.
- Cache invalidation and replica lag are common interview traps.
- Shard key choice is one of the most important design decisions.

Common Comparisons

Compare	Main Difference
Horizontal vs vertical scaling	Add machines vs upgrade machine
Stateful vs stateless	Local session vs no local session
Cache vs database	Fast temporary layer vs source of truth
Replication vs sharding	Copy data vs split data
Synchronous vs asynchronous replication	Wait for replica vs replicate later

Must-Remember Facts

- Read replicas do not solve high write load.
- Caches can return stale data.
- Sharding is powerful but operationally complex.
- Bad shard keys create hot shards.

- Stateless app servers are easier to autoscale.
- Load balancers need health checks.
- Scaling requires monitoring and capacity planning.

Interview Traps

- Do not say "just add cache" without invalidation.
- Do not say "just shard" without shard key.
- Do not ignore consistency.
- Do not forget failure cases.
- Do not confuse high availability with high performance.

13. Practice Tasks

Task 1: Design A Scaled URL Shortener

Draw a design for a URL shortener that handles millions of redirects per day.

Include:

- Load balancer
- Stateless app servers
- Cache for short-code lookup
- Primary database
- Read replicas
- Sharding strategy for short codes

Think about:

- What should be cached?
- What is the shard key?
- What happens if a short URL is not found?

Task 2: Identify Bottlenecks

Given this system:

```
Client -> App Server -> Database
```

Traffic increases 10x.

Write down:

- What metrics you would check first.
- Whether the bottleneck is CPU, memory, network, or database.
- What scaling technique you would apply first.

Task 3: Cache-Aside Simulation

Write pseudocode for fetching product details using cache-aside:

```
getProduct(productId):  
  check Redis  
  if found, return  
  query DB  
  store in Redis with TTL  
  return
```

Then answer:

- What happens during cache miss?
- How do you invalidate cache when product price changes?
- What TTL would you choose?

Task 4: Shard Key Selection

Choose a shard key for each system:

System	Possible Shard Key
Chat app	<code>conversation_id</code>
Food delivery	<code>city_id</code> or <code>restaurant_id</code>
Banking app	<code>account_id</code>
Social media posts	<code>user_id</code> or post ID hash
Analytics events	hash of user ID plus time partition

Explain why your key works and what hot spot risks it has.

Task 5: Replication Lag Scenario

A user updates their profile name from "Amit" to "Amit Kumar". Immediately after saving, the profile page still shows "Amit".

Explain:

- Why this happens.
- Which component is stale.
- How to fix it for important flows.

Task 6: Draw Scaling Stages

Draw the evolution:

```
Stage 1: One server, one DB  
Stage 2: Load balancer + multiple app servers  
Stage 3: Add Redis cache
```

Stage 4: Add read replicas
Stage 5: Add sharding

For each stage, write what bottleneck it solves.

Task 7: Mini Python Cache Example

```
import time

cache = {}

def get_from_db(key):
    print("DB call")
    return f"value_for_{key}"

def get_value(key, ttl_seconds=5):
    now = time.time()

    if key in cache:
        value, expiry = cache[key]
        if now < expiry:
            return value

    value = get_from_db(key)
    cache[key] = (value, now + ttl_seconds)
    return value

print(get_value("product:101"))
print(get_value("product:101"))
```

Expected observation:

The first call hits the database. The second call returns from cache.

14. Final Cheat Sheet

Core Definition

Scaling is increasing a system's ability to handle more traffic, data, users, or operations.

Why It Matters

Scaling keeps systems fast, available, and reliable as usage grows.

Most Asked Questions

- What is horizontal scaling?
- What is vertical scaling?
- Why should services be stateless?
- What is caching?

- What is cache invalidation?
- What is replication?
- What is replica lag?
- What is sharding?
- How do you choose a shard key?
- Replication vs sharding?

Common Comparisons

Comparison	One-Line Difference
Horizontal vs vertical scaling	Add machines vs upgrade one machine
Stateful vs stateless	Local session storage vs external/no local state
Cache vs database	Fast temporary access vs durable source of truth
Replication vs sharding	Copy data vs split data
Read scaling vs write scaling	Replicas/cache vs sharding/partitioning

One-Line Interview Answer

Scaling means growing a system's capacity; we scale app servers horizontally using load balancers and stateless services, reduce repeated work using caching, scale database reads using replication, and scale large data or heavy writes using sharding.